Learning outcomes:

1. mergesort, quicksort, với mỗi thuật toán:

- biết cách chạy thuật toán với dữ liệu cụ thể (viết được vết chạy)
- biết độ phức tạp về thời gian chạy tính theo số phép so sánh, số phép đổi chỗ, trong các trường hợp tốt nhất, trung bình, tồi nhất
- có thể chọn thuật toán phù hợp với đặc điểm dữ liệu đâù vào (ví dụ mảng sắp ngược, đã sắp xếp, gần sắp xếp, nhiều khóa trùng nhau…)
- với quicksort cần hiểu được vai trò của bước tráo ngẫu nhiên (hoặc pivot ngẫu nhiên) trong việc tránh trường hợp xấu nhất.

2. biết các tiêu chí sắp xếp **tại chỗ (in-place)**, sắp xếp **ổn định (stable)**, và đánh giá được các thuật toán sắp xếp đã học theo các tiêu chí này

bonus: quick select, 3-way quicksort

Demo các thuật toán sắp xếp: https://visualgo.net/en/sorting
Xem các thuật toán Mergesort, Quicksort

Project để đề phòng sơ xảy điểm chuyên cần hoặc giữa kì

Mergesort và Quicksort có chung một cải tiến là CUTOFF dùng Insertion sort đối với mảng độ dài nhỏ. Các phiên bản giáo trình nhắc đến độ dài CUTOFF 7 (cũ) và 10 (mới). Làm sao biết được giá trị nào thì phù hợp ? **Làm thí nghiệm và đo thử.**
Bạn hãy làm thí nghiệm tìm giá trị CUTOFF tốt nhất đối với cấu hình máy của bạn và cài đặt mergesort mà bạn dựng từ nội dung bài giảng.
Bạn được dùng AI để thảo luận, viết script chạy thí nghiệm, xây dựng bộ dữ liệu test. Bạn cần tự mình dựng cài đặt mergesort **từ nội dung bài giảng**, không dùng các cài đặt khác. Xem lại chủ đề 1.3. Algorithm Analysis để ôn tập về thí nghiệm và thống kê kết quả thí nghiệm.

Sản phẩm nộp gồm có:

- Báo cáo thí nghiệm: mô tả thí nghiệm, phân tích tổng kết kết quả (lập bảng thống kê, vẽ đồ thị…).
- Cài đặt mergesort và các script chạy thí nghiệm
- bộ dữ liệu thí nghiệm
- log chat với AI (nếu có)

Deadline: sau thi giữa kì sẽ quyết định.

1. COS226, Midterm f25, 3

Cột bên trái nhất chứa một mảng gồm 24 số nguyên cần sắp xếp. Cột phải nhất chứa mảng kết quả đã được sắp xếp. Các cột khác là nội dung của mảng tại một bước trung gian nào đó trong quá trình chạy của một trong năm thuật toán sắp xếp. Hãy cho biết cột nào là bước trung gian của thuật toán sắp xếp nào, điền chữ cái tương ứng vào ô vuông dưới mỗi cột

37	11	11	95	35	11	11
11	17	20	94	11	16	16
39	20	23	83	16	17	17
86	23	35	88	17	20	20
54	35	37	92	20	23	23
44	37	39	68	23	35	35
35	39	44	37	37	37	37
23	44	54	75	44	39	39
94	54	67	86	94	40	40
20	67	68	56	54	42	42
67	68	86	67	67	44	44
68	83	94	63	68	54	54
95	86	17	44	95	95	56
17	88	75	17	86	68	63
83	94	77	35	83	83	67
88	95	83	23	88	88	68
75	75	88	11	75	75	75
77	77	95	77	77	77	77
40	40	16	40	40	94	83
42	42	40	42	42	86	86
56	56	42	20	56	56	88
92	92	56	54	92	92	92
16	16	63	16	39	67	94
63	63	92	39	63	63	95
A						G

A, mảng gốc
B, Selection sort
C, Insertion sort
D, Mergesort (top-down)
E, Quicksort (chuẩn, không tráo ngẫu nhiên)
F, Heapsort
G, Mảng đã sắp xếp

2. COS226, Midterm f25, 4

Xét thuật toán sắp xếp sau:

```
public static void mysterySort(Comparable[] a) {
    int n = a.length;
```

```
        for (int i = 0; i < n; i++)
            { boolean
            exchanged = false;
            for (int j = 0; j < n - i - 1; j++) { if
                (less(a[j+1], a[j])) {
                exch(a, j+1, j);
                exchanged =
                true;
            }
        }
        if (!exchanged) break;          // no exchanges, already sorted
    }
}
```

a, Bất biến nào đúng tại điểm bắt đầu của mỗi lần lặp i (với mỗi giá trị của i)? Chọn tất cả các phát biểu đúng

- ☐ i phần tử đầu tiên trong mảng đã được sắp xếp đúng thứ tự
- ☐ i phần tử cuối cùng trong mảng đã được sắp xếp đúng thứ tự
- ☐ i phần tử đầu tiên trong mảng là i phần tử nhỏ nhất của mảng
- ☐ i phần tử cuối cùng trong mảng là i phần tử lớn nhất của mảng

b, thuật toán trên có phải thuật toán sắp xếp tại chỗ (in-place) hay không?
c, thuật toán trên có phải thuật toán sắp xếp ổn định (stable) hay không?
d, Giả sử ta chạy mysterySort() với input là mảng a[] độ dài n. Hãy ghép câu hỏi bên trái với câu trả lời đúng bên phải

• Số lần gọi hàm exch() để sắp xếp một mảng số nguyên có dạng [2,3,4,..n-1, n]	A. 0 B. ~½ n C. ~n D. ~2n E. ~½ n log n F. ~n log n G. ~¼ n^2 H. ~½ n^2 I. ~n^2
• Số lần gọi less() trong trường hợp tốt nhất (best case)	
• Số lần gọi less() trong trường hợp tồi nhất (worst case)	

3. COS226, Midterm s25, 3

Cột bên trái nhất chứa một mảng gồm 24 số nguyên cần sắp xếp. Cột phải nhất chứa mảng kết quả đã được sắp xếp. Các cột khác là nội dung của mảng tại một bước trung gian nào đó trong quá trình chạy của một trong năm thuật toán sắp xếp. Hãy cho biết cột nào là bước trung gian của thuật toán sắp xếp nào, điền chữ cái tương ứng vào ô vuông dưới mỗi cột

66	45	11	11	37	11	11
65	37	14	13	65	13	13
22	41	19	14	22	14	14
19	35	22	16	19	16	16
46	30	30	19	46	19	19
30	22	35	22	30	22	22
11	26	46	26	11	26	26
35	13	65	30	35	30	30
79	11	66	35	41	35	35
99	14	68	37	61	46	37
14	16	79	41	14	50	41
68	19	99	45	45	65	45
50	46	13	50	50	66	46
26	50	16	66	26	68	50
13	61	26	65	13	79	61
16	65	37	46	16	99	65
78	66	41	78	66	78	66
37	68	45	99	78	37	68
93	72	50	93	93	93	72
45	78	61	68	68	45	78
85	79	72	85	85	85	79
61	85	78	61	99	61	85
72	93	85	72	72	72	93
41	99	93	79	79	41	99
A						G

A, mảng gốc
B, Selection sort
C, Insertion sort
D, Mergesort (top-down)
E, Quicksort (chuẩn, không tráo ngẫu nhiên)
F, Heapsort
G, Mảng đã sắp xếp

4. COS226, Midterm f24, 3

Cột bên trái nhất chứa một mảng gồm 24 số nguyên cần sắp xếp. Cột phải nhất chứa mảng kết quả đã được sắp xếp. Các cột khác là nội dung của mảng tại một bước trung gian nào đó trong quá trình chạy của một trong năm thuật toán sắp xếp. Hãy cho biết cột nào là bước trung gian của thuật toán sắp xếp nào, điền chữ cái tương ứng vào ô vuông dưới mỗi cột

49	14	14	48	14	57	14
45	22	22	45	15	56	15
14	45	24	14	22	49	22
97	49	37	15	24	45	24
22	57	40	22	37	40	37
57	97	44	44	40	37	40
40	24	45	40	44	48	44
24	40	48	24	45	14	45
65	63	49	37	48	44	48
93	65	57	49	49	24	49
63	68	63	63	56	15	56
68	93	65	68	57	22	57
75	75	68	75	75	58	58
48	48	75	93	65	60	60
37	37	93	65	93	63	63
44	44	97	57	68	64	64
73	73	73	73	73	65	65
15	15	15	97	97	68	68
58	58	58	58	58	73	73
64	64	64	64	64	75	75
88	88	88	88	88	78	78
60	60	60	60	60	88	88
56	56	56	56	63	93	93
78	78	78	78	78	97	97
A						G

A, mảng gốc
B, Selection sort
C, Insertion sort
D, Mergesort (top-down)
E, Quicksort (chuẩn, không tráo ngẫu nhiên)
F, Heapsort
G, Mảng đã sắp xếp

5. COS226, Midterm s25, 4

Xét một mảng 2n phần tử ở dạng 1, n+1, 2, n+2, 3, n+3, …, n, 2n
Ví dụ với n=8 mảng có nội dung như sau: 1 9 2 10 3 11 4 12 5 13 6 14 7 15 8 16
Hỏi mỗi thuật toán sắp xếp dưới đây dùng đến bao nhiêu phép so sánh, tính theo n? Chú ý rằng mảng có độ dài 2n, không phải n.

Selection sort: ~ ¼ n^2 ~ ½ n^2 ~ n^2 ~¼ n^2 ~ 3/2 n^2 ~ 2 n^2
Insertion sort: ~ ¼ n^2 ~ ½ n^2 ~ n^2 ~¼ n^2 ~ 3/2 n^2 ~ 2 n^2
Mergesort: ~ ¼ n log n ~ ½ log n ~ n log n ~ 3/2 n log n ~ 2 n log n

6. COS226, Midterm f24, 7

Giả sử n là một lũy thừa của 2. Xét một mảng có cấu trúc như sau

2n 2n … 2n 0 1 2 2 4 4 4 4 8 8 8 8 8 8 8 8 … n n … n

Nửa đầu của mảng chứa 2n lần lặp lại của giá trị 2n. Nửa sau chứa 2n số: một số 0, một số 1, hai lần lặp lại của số 2, bốn lần lặp lại của số 4, tám lần lặp lại của số 8, mười sáu lần lặp lại của số 16, và cứ tiếp tục như vậy cho đến khi số n xuất hiện n lần.

Ví dụ, với n = 4 mảng có nội dung sau

8 8 8 8 8 8 8 8 0 1 2 2 4 4 4 4

Mỗi thuật toán sắp xếp dưới đây (thuật toán chuẩn, từ giáo trình) thực hiện bao nhiêu phép so sánh dưới dạng một hàm của n trong trường hợp xấu nhất? Lưu ý rằng độ dài của mảng là 4n chứ không phải n.

Chú ý: Ta viết log để biểu thị logarit cơ số 2 và nhắc lại rằng log(ab) = log a + log b.

a. Selection sort:

   ~ ½ n^2 ~ n^2 ~$2n^2$ ~$4n^2$ ~$8n^2$ ~$16n^2$

b. Insertion sort:

   ~ ½ n^2 ~ n^2 ~$2n^2$ ~$4n^2$ ~$8n^2$ ~$16n^2$

c. Mergesort:

   ~ n log n ~2n log n ~4n log n ~$\frac{9}{2}$ n log n ~8n log n ~16n log n

d. 3-way quicksort: Giả sử bước tráo ngẫu nhiên cho kết quả là mảng sắp xếp giảm dần

   2n 2n … 2n n n … n …8 8 8 8 8 8 8 8 4 4 4 4 2 2 1 0

   $\Theta(n)$ $\Theta(n \log n)$ $\Theta(n^2 / \log n)$ $\Theta(n^2)$ $\Theta(n^2 \log n)$ $\Theta(n^4)$

Bài 1 Intro to Tutorial Challenges (on Sorting).

Tóm tắt: Tìm kiếm một số trong dãy đã được sắp tăng dần. In ra chỉ số tìm được.
Sử dụng tìm kiếm nhị phân (binary search – đã có từ bài tập BT4 và mã nguồn có trong slide bài giảng) cho bài này.

Bài 2 – Khảo sát MergeSort

Thực hiện tương tự với bài trên, thay Insertion Sort bằng Mergesort. Dùng cùng bộ dữ liệu với Insertion sort và so sánh thời gian chạy với Insertion Sort tại từng loại dữ liệu và kích thước dữ liệu

Bài 3 – Khảo sát QuickSort

Thực hiện tương tự với bài trên, nhưng thực hiện trên QuickSort với hai phiên bản: (1) có shuffle như trong bài giảng, (2) không có shuffle. Dùng cùng bộ dữ liệu và so sánh thời gian chạy với các thuật toán Insertion Sort và Merge Sort khác tại từng loại dữ liệu và kích thước dữ liệu

Bài 5 Quicksort 1 – Partition

Tóm tắt: Viết thêm vào thủ tục partition(int[] ar) để chia mảng thành 2 phần nhỏ hơn và lớn hơn phần tử chốt (được chọn là đầu của mảng). In kết quả sau phân chia.
Đã có sẵn phần code nhập dữ liệu đầu vào, và in mảng ra printArray(int[] ar).

Bài 6 Quicksort In-Place

Tóm tắt: Cài đặt thuật toán QuickSort. In ra toàn bộ mảng các số cứ sau mỗi lần thực hiện partition (mỗi lần phân chia).
Chú ý: Việc partition ở đây cần phải sử dụng phương pháp Lomuto Partitioning. Phương pháp này đơn giản hơn và chậm hơn phương pháp của Hoare học trên lớp (xem tại Wikipedia). Mã của Lomuto Partitioning như sau:

```
algorithm partition(A, low, high) is
    pivot := A[high]
    i := low        // place for swapping
    for j := low to high – 1 do
        if A[j] ≤ pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[high]
    return i
```

Bài 7 Find the Median

Tóm tắt: Tìm phần tử trung vị. Với dãy có (2n + 1) số, thì phần tử trung vị có n số nhỏ hơn (hoặc bằng) nó, và đồng thời có n số lớn hơn (hoặc bằng) nó.
Gợi ý: Không cần sử dụng sắp xếp. Thay vào đó sử dụng việc phân chia (partition) như của QuickSort. Chia dãy thành 2 phần, phần tử trung vị ở một trong 2 nửa này. Đệ qui để tìm trong nửa đó.
Hãy phân tích độ phức tạp tích toán của thuật toán trên.

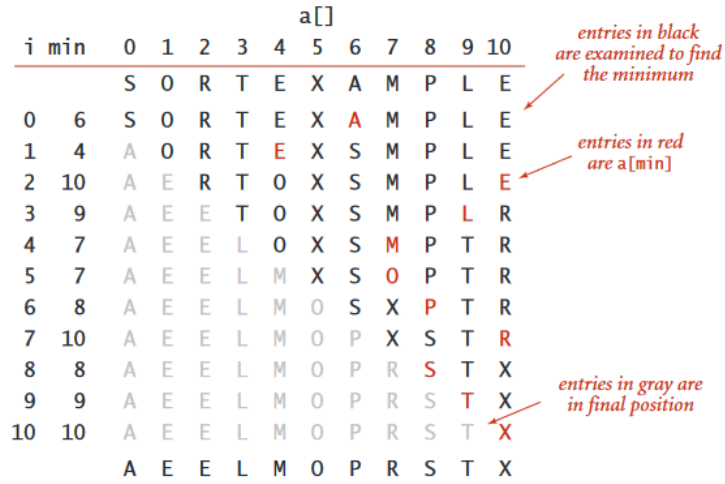
Bài 8 – Hàng đợi ưu tiên.

Bài tập này để chuẩn bị cho chủ đề tiếp theo trong chương trình
Hãy cài đặt một cấu trúc dữ liệu hàng đợi ưu tiên với các hàm sau:

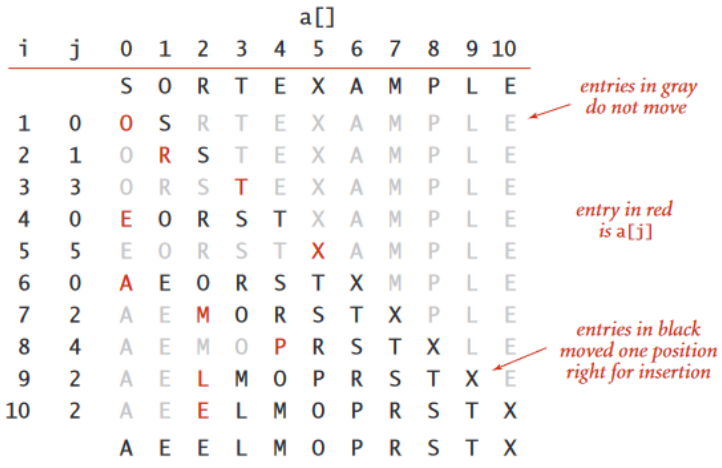
```
class PriorityQueue {
   …
   void insert(int);    //  thêm một số vào hàng đợi
   int deleteMin();    // xóa số nhỏ nhất ra khỏi hàng đợi và trả về số đó.
}
```

Gợi ý: Đây không phải lời giải tối ưu, nhưng bạn có thể tạm giả thiết hàng đợi chỉ chứa tối đa K phần tử và dùng mảng kích thước K để cài cấu trúc trên. Dùng thêm 1 biến để lưu trạng thái hiện tại Q đang chứa bao nhiêu số. Khi nói rằng hàng đợi đang chứa M số có nghĩa là các phần tử mảng chỉ số từ 0 tới M-1 đang là M số đó, các phần tử mảng chỉ số >= M không phải quan tâm đến.

2.1.1 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán selection sort.
Gợi ý: trình bày tương tự ví dụ dưới đây. Các phần tử mầu đen được đọc giá trị để tìm giá trị min, các phần tử mầu đỏ là a[min], mầu xám là các phần tử đã được xếp vào đúng vị trí cuối cùng.



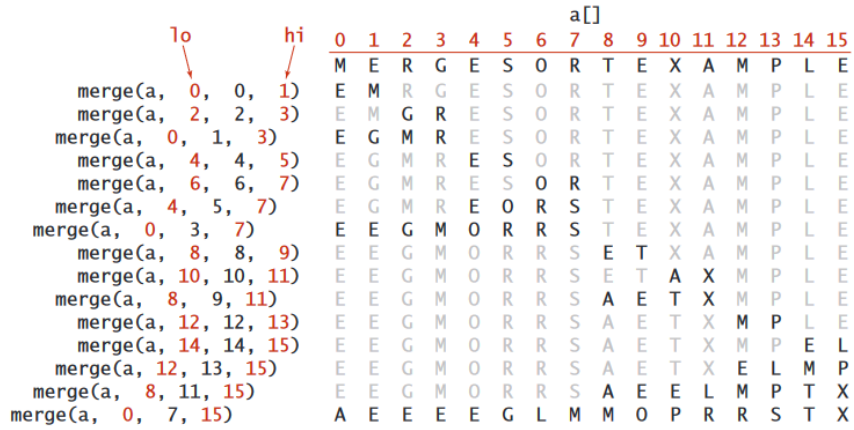
2.1.2 Trong thuật toán selection sort, một phần tử bất kì cần đến tối đa bao nhiêu lần đổi chỗ để được sắp vào đúng chỗ? Một phần tử bất kì cần đến trung bình bao nhiêu lần đổi chỗ để được sắp vào đúng chỗ?
2.1.3 Cho ví dụ về một mảng N phần tử mà khi chạy thuật toán selection sort có số lần kiểm tra `a[j] < a[min]` ra false (và vì thế cập nhật giá trị `min`) là tối đa.
2.1.4 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán insertion sort.
Gợi ý: trình bày tương tự ví dụ dưới đây. Các phần tử mầu đen di chuyển sang phải 01 vị trí để thực hiện việc chèn, phần tử mầu đỏ là a[j], mầu xám là các phần tử không di chuyển.



2.1.5 Với mỗi điều kiện trong hai điều kiện của vòng for bên trong của thuật toán insertion sort, hãy mô tả một mảng N phần tử mà điều kiện đó luôn false khi vòng lặp kết thúc.
2.1.6 Thuật toán nào chạy nhanh hơn đối với mảng gồm tất cả các key bằng nhau, selection sort hay insertion sort?
2.1.7 Thuật toán nào chạy nhanh hơn cho mảng đã sắp xếp thứ tự ngược, selection sort hay insertion sort?
2.1.8 (*) Giả sử ta dùng thuật toán insertion sort đối với một mảng thứ tự ngẫu nhiên mà trong đó các phần tử chỉ nhận một trong 03 giá trị. Hỏi thời gian chạy sẽ là tuyến tính, bậc hai, hay cái gì đó ở giữa?

2.2.Mergesort

2.2.2 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán mergesort top-down.
Gợi ý: trình bày tương tự ví dụ dưới đây.

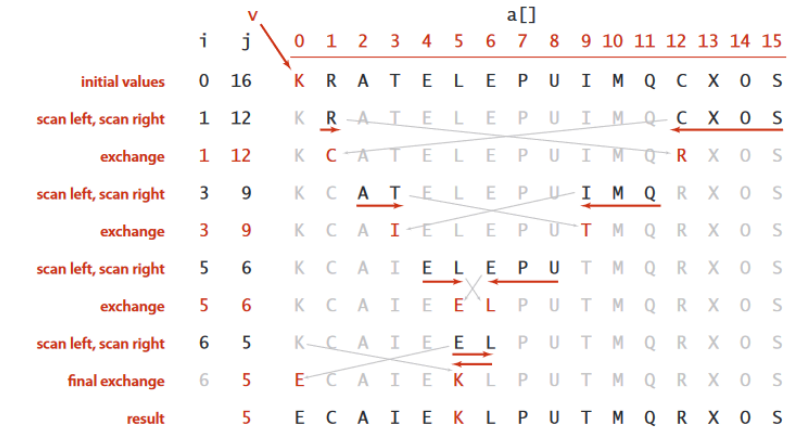


2.2.3 Tương tự câu 2.2.2 nhưng sử dụng thuật toán bottom-up mergesort.
2.2.5 Cho N = 39, hỏi thuật toán mergesort top-down chạy chuỗi hàm merge với các kích thước mảng nào? Tương tự với bottom-up mergesort.
2.2.7 (*) Chứng minh rằng số bước so sánh mà mergesort dùng đến tăng dần đều (C(N+1) > C(N) với N > 0).
2.2.8 (*) Giả sử thuật toán mergesort top-down được sửa để bỏ qua lệnh `merge()` mỗi khi `a[mid] <= a[mid+1]`. Hãy chứng minh rằng số phép so sánh dùng để sắp xếp một mảng đã được sắp sếp là tuyến tính.

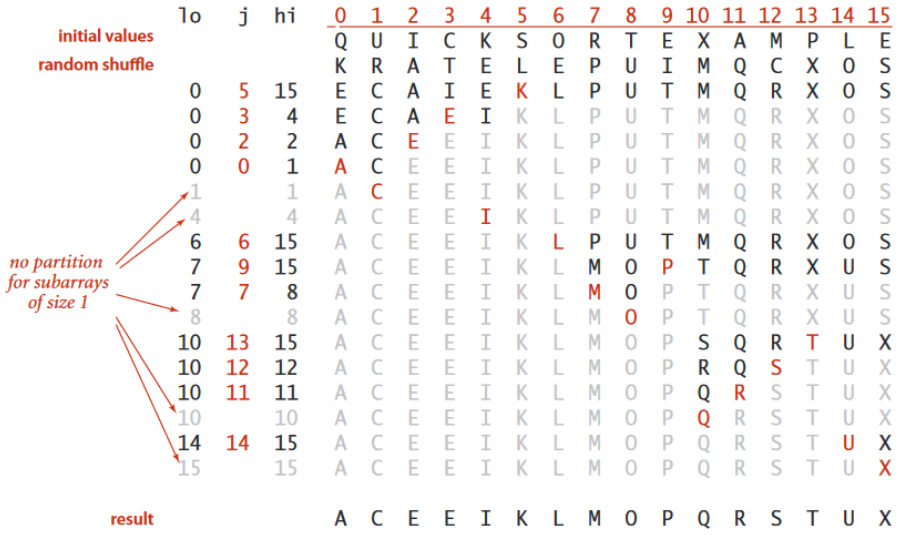
N/8 nút 8
N/4 nút 4 4
N/2 nút 2 2 2 ……
N nút 1 1 1 1 1 1…..
N + N/2 + N/4 + ….. 1 ~2N

2.3. Quicksort

2.3.1 Trình bày trình tự hàm `partition()` phân hoạch mảng `E A S Y Q U E S T I O N`.
Gợi ý: trình bày tương tự ví dụ dưới đây: vết chạy của phân hoạch, nội dung mảng trước và sau mỗi lần đổi chỗ.



2.3.2 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán quicksort bỏ qua bước tráo ngẫu nhiên khởi đầu. Gợi ý cách trình bày tương tự như dưới đây.



2.3.3 (*) Khi chạy quicksort cho một mảng độ dài N, phần tử lớn nhất có thể bị đổi chỗ tối đa bao nhiêu lần?

2.3.4 Giả sử ta bỏ qua bước tráo ngẫu nhiên ban đầu. Hãy cho 6 mảng độ dài 10 mà quicksort sẽ dùng số phép so sánh nhiều nhất (worst-case của thời gian chạy nếu tính theo số phép so sánh).

2.3.5 Hãy viết thuật toán sắp xếp một mảng được biết trước là các phần tử chỉ có một trong 2 giá trị.

2.3.7 (*) Khi quicksort được dùng để sắp xếp một mảng gồm N phần tử có giá trị phân biệt, hỏi trung bình có bao nhiêu lần quicksort chạy với mảng kích thước 0, 1 và 2.

2.3.8 Hỏi quicksort dùng đến bao nhiêu phép so sánh nếu sắp xếp một mảng gồm N phần tử có giá trị bằng nhau?

2.3.9 (*) Giải thích hiện tượng xảy ra khi chạy quicksort cho mảng gồm các phần tử chỉ có 2 giá trị phân biệt. Câu hỏi tương tự với 3 giá trị phân biệt.

2.3.11 (*) Giả sử ta dùng thuật toán phân hoạch mà bỏ qua các phần tử có giá trị bằng giá trị phần tử phân hoạch thay vì dừng lại. Hãy chứng minh rằng thời gian chạy của quicksort với hàm phân hoạch này sẽ là bậc hai với tất cả các mảng mà số giá trị phân biệt của các phần tử mảng chỉ là một hằng số.

2.3.13 (*) Độ sâu gọi đệ quy của quicksort là bao nhiêu trong trường hợp tốt nhất, tồi nhất, trung bình?